

Team Structure & Task Allocation (12-Hour Hackathon)



Developer 1 — Blockchain Lead & Technical Architect

Primary Objective

Own everything related to Avalanche and ensure the blockchain layer is simple, reliable, and easy for the team to consume.

Tasks

- Design and develop the `RiderRewardHub.sol` smart contract.
- Deploy the contract to Avalanche Fuji Testnet.
- Create helper functions for reward distribution.
- Generate and share:
 - Contract ABI
 - Contract Address
 - Environment variables
- Create a simple backend helper:
`distributeReward(walletAddress, amount, reason)`
- Test all blockchain interactions before integration.
- Assist with technical decisions across the team.

Deliverables

- Deployed smart contract.
- Verified contract address.
- ABI file.
- Reward helper.
- Working on-chain reward transaction.

Notes

- Keep contracts simple.
- Don't over-engineer tokenomics.
- Prioritize reliability over features.
- If blockchain becomes a blocker, provide mocked responses so the rest of the team keeps moving.



Developer 2 — Backend & Gamification Engine

Primary Objective

Build the brain of the application.

This developer owns the game logic—not just the API.

Tasks

- Set up Supabase.
- Create database schema.
- Build REST API.
- Design XP system.
- Implement scoring engine.
- Implement level progression.
- Implement mission progression.
- Generate player feedback.
- Determine reward eligibility.
- Trigger blockchain rewards when milestones are reached.

Deliverables

- Working backend API.
- XP calculations.
- Level system.
- Mission engine.
- Feedback engine.
- Reward trigger.

Notes

Every API response should answer:

- How did the player perform?
- What did they earn?
- What should they improve next?

Think like a game designer.



Developer 3 — Frontend & Player Experience

Primary Objective

Build the player's world.

The dashboard should feel like a game—not an admin panel.

Tasks

- Set up React + Vite.
- Configure Tailwind CSS.
- Build Rider Dashboard.
- Build Mission Panel.
- Build XP Progress Bar.
- Build Level Progression UI.
- Build Achievement Popups.
- Build Feedback Cards.
- Build Reward Notifications.
- Create Delivery Simulator buttons.
- Display transaction status.

Deliverables

- Responsive dashboard.
- Interactive delivery simulator.
- Visual mission progress.
- Real-time XP updates.
- Animated level-up experience.

Notes

The UI should always answer:

- How am I doing?
- What's my current mission?
- How close am I to leveling up?
- What should I do next?

Avoid unnecessary polish until the complete player loop works.



Developer 4 — Integration, QA & Demo Lead

Primary Objective

Own the complete player journey from end to end.

This developer ensures every component works together and that the judges experience a seamless demo.

Tasks

- Implement wallet connection.
- Connect frontend to backend.
- Connect backend to Avalanche.
- Test all API endpoints.
- Handle loading and error states.
- Create Demo Mode.
- Prepare presentation.
- Write demo script.
- Record backup demo video.
- Perform final QA.

Deliverables

- Working wallet integration.
- Fully integrated application.
- Demo-ready experience.
- Presentation slides.
- Backup video.

Notes

This role becomes increasingly important as the hackathon progresses.

Own:

- Integration.

- Testing.
- Bug fixing.
- Demo confidence.

If anything breaks during judging, this developer is the first responder.

Team Collaboration Rules

First 30 Minutes

- Agree on the player journey.
 - Freeze the API contract.
 - Freeze the database schema.
 - Define scoring rules.
 - Define missions.
 - Set up repositories and Git workflow.
-

Shared API Contract

No developer should change the API without team agreement.

Everyone builds against the same response structure.

Git Workflow

- `main`
- `develop`
- `feature/blockchain`
- `feature/backend`
- `feature/frontend`
- `feature/integration`

Merge frequently into `develop`.

Hourly Check-ins (5 Minutes)

Each developer answers:

- What did I finish?
- What is blocking me?
- What do I need?

Keep updates short.

Last Two Hours

Stop adding features.

Focus on:

- Integration.
 - Testing.
 - Bug fixes.
 - Demo flow.
-

Last Hour

No new code unless fixing critical issues.

Practice the demo repeatedly until everyone knows their role.

MVP Success Criteria

The demo should complete this journey in under 90 seconds:

1. Connect wallet.
2. Open Rider Dashboard.
3. View current mission.
4. Simulate a completed delivery.
5. Receive immediate feedback.
6. Earn XP.

7. Progress toward the next level.
8. Unlock a reward.
9. Trigger an Avalanche transaction.
10. Refresh the dashboard with updated progress.

If this loop works smoothly, the MVP has achieved its objective.

Overall Notes

- Build **one complete experience**, not four separate components.
- Focus on **player motivation**, not feature count.
- If blocked, **mock dependencies** instead of waiting.
- Blockchain should support the game loop, not dominate it.
- Every screen and API should reinforce the player's journey:
Challenge → Action → Feedback → Progress → Reward → Next Challenge.
- Optimize for a **reliable demo** over a feature-rich product. A polished end-to-end flow will outperform a larger but incomplete system.